

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «ОМО»
Тема: «Нелинейные ИНС в задачах регрессии»

Выполнил:
Студент 3-го курса
Группы АС-66
Осовец А.О.
Проверил:
Крощенко А.А.

Брест 2025

Вариант 7

1. Выполнить моделирование прогнозирующей нелинейной ИНС. Для генерации обучающих и тестовых данных использовать функцию. Для прогнозирования использовать многослойную ИНС с одним скрытым слоем. В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

№ варианта	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое
1	0.1	0.1	0.05	0.1	6	2
2	0.2	0.2	0.06	0.2	8	3
3	0.3	0.3	0.07	0.3	10	4
4	0.4	0.4	0.08	0.4	6	2
5	0.1	0.5	0.09	0.5	8	3
6	0.2	0.6	0.05	0.6	10	4
7	0.3	0.1	0.06	0.1	6	2
8	0.4	0.2	0.07	0.2	8	3
9	0.1	0.3	0.08	0.3	10	4
10	0.2	0.4	0.09	0.4	6	2
11	0.3	0.5	0.05	0.5	8	3

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd
```

1. Генерация обучающих и тестовых данных

```
def target_function(x, a=0.3, b=0.1, c=0.06, d=0.1):
    return a * torch.cos(b * x) + c * torch.sin(d * x)
```

```
a, b, c, d = 0.3, 0.1, 0.06, 0.1
input_size = 6
hidden_size = 2
num_samples = 200
train_ratio = 0.8
```

```
x = torch.linspace(0, 10, num_samples).reshape(-1, 1)
X = torch.cat([x ** i for i in range(1, input_size + 1)], dim=1)
y = target_function(x, a, b, c, d)
```

```
split_idx = int(num_samples * train_ratio)
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]
```

```
print("Данные сгенерированы для функции  $y = a \cdot \cos(bx) + c \cdot \sin(dx)$ ")
print(f"Параметры: a={a}, b={b}, c={c}, d={d}")
print(f"Размер обучающей выборки: {X_train.shape}, тестовой: {X_test.shape}")
```

2. Архитектура ИНС

```

class NonlinearRegressor(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.hidden = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Sigmoid()
        )
        self.output = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        x = self.hidden(x)
        return self.output(x)

model = NonlinearRegressor(input_size, hidden_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

```

3. Обучение модели

```

losses = []
epochs = 500

```

```

for epoch in range(epochs):
    model.train()
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

```

```

print("\nОбучение завершено")
print(f"Минимальная ошибка достигнута при a={a}, финальный MSE={min(losses):.6f}")

```

4. График прогнозируемой функции на обучающем участке

```

model.eval()
with torch.no_grad():
    y_train_pred = model(X_train)

plt.figure(figsize=(10, 4))
plt.plot(x[:split_idx], y_train, label="Эталон")
plt.plot(x[:split_idx], y_train_pred, label="Прогноз")
plt.title("Прогнозируемая функция на обучающем участке")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

```

5. Результаты обучения: таблица и график ошибки

```

train_table = pd.DataFrame({
    "Эталонное значение": y_train.squeeze().numpy(),
    "Полученное значение": y_train_pred.squeeze().numpy(),
    "Отклонение": (y_train_pred - y_train).squeeze().numpy()
})
print("\nРезультаты обучения (первые строки):")
print(train_table.head())

```

```

plt.figure(figsize=(10, 4))
plt.plot(losses)
plt.title("График изменения ошибки по эпохам")
plt.xlabel("Эпоха")
plt.ylabel("MSE")
plt.grid(True)
plt.show()

```

6. Результаты прогнозирования

```

with torch.no_grad():
    y_test_pred = model(X_test)
    test_table = pd.DataFrame({
        "Эталонное значение": y_test.squeeze().numpy(),
        "Полученное значение": y_test_pred.squeeze().numpy(),
        "Отклонение": (y_test_pred - y_test).squeeze().numpy()
    })
print("\nРезультаты прогнозирования (первые строки):")
print(test_table.head())

```

Результат:

Данные сгенерированы для функции $y = a \cdot \cos(bx) + c \cdot \sin(dx)$

Параметры: $a=0.3$, $b=0.1$, $c=0.06$, $d=0.1$

Размер обучающей выборки: `torch.Size([160, 6])`, тестовой: `torch.Size([40, 6])`

Результаты обучения (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.300000	0.293966	-0.006034
1	0.300298	0.294472	-0.005825
2	0.300588	0.295011	-0.005577
3	0.300870	0.295602	-0.005268
4	0.301145	0.296260	-0.004886

Результаты прогнозирования (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.251354	0.287043	0.035689
1	0.250475	0.287043	0.036568
2	0.249589	0.287043	0.037454
3	0.248696	0.287043	0.038347
4	0.247798	0.287043	0.039245

